

PMSCS 622P

Programming Languages

Lecture 04: Loops

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Lecture Roadmap

■ I. Concepts & Syntax

- **Mechanics:** The 4 Pillars of a Loop
- **The Big 3:** while, for, do-while
- **Control:** break and continue

■ II. Analysis & Practice

- **Patterns:** Accumulators & Searching
- **Complexity:** $O(N)$ vs $O(\log N)$ vs $O(N^2)$

The Problem: Manual Repetition

Imagine you need to print numbers 1 to 1000.

The Naive Way:

```
cout << 1;  
cout << 2;  
cout << 3;  
// ... Copy-paste 996 times ...  
cout << 1000;
```

Why this fails:

- X Redundant:** Code size explodes.
- X Rigid:** What if we need 1 to N ?
- X Error-Prone:** Hard to debug.

The Solution: Loops

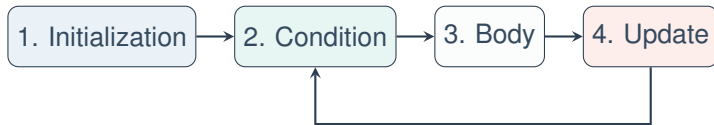
Definition

A **Loop** is a control structure that repeats a block of code as long as a specified condition is True.

Core Idea: Write the code **once**, execute it **many times**.

The 4 Pillars of a Loop

Every loop needs these four sections to function correctly.



1. Initialization

Role: Sets the **starting point**. Executed **only once** before the loop begins.

```
| int i = 0; // Start counter at 0
```

Risk: Garbage Data

If you forget to initialize, the variable will hold random memory values (garbage), causing unpredictable bugs.

2. Condition

Role: The "Gatekeeper". Checked **before** (or after) every single iteration.

```
| while (i < 10) // Evaluates to Boolean (True/False)
```

If True ✓

Gate Opens.
Execute the **Body**.

If False ✕

Gate Closes.
Terminate the loop immediately.

3. Body

Role: The "Payload". The actual logic you want to repeat.

```
{  
    cout << "Student ID: " << i;  
    process_grades(i);  
}
```

Scope & Scale

This block can contain a single line or thousands of lines (including other loops!).

Tip: In C++/Java, the curly braces { } define the scope of this body.

4. Update

Role: The "Propulsion". Moves the loop variable toward the termination condition.

```
i++;           // Standard increment  
// OR  
i = i + 2;     // Step by 2  
i--;           // Countdown
```

Critical Error: The Infinite Loop

If you forget this step, the Condition will **never** become False. The program will hang forever.

The 'while' Loop: Check, Then Act

Concept: The simplest loop. It repeats code **as long as** a condition remains True. *Think: "While it is raining, keep umbrella open."*

C++ / Java Syntax

```
int hunger = 5;

while (hunger > 0) {
    eat_cookie();
    hunger--;
}
```

Key: Uses () for condition and { } for scope.

Python Syntax

```
hunger = 5

while hunger > 0:
    eat_cookie()
    hunger -= 1
```

Key: No braces. Uses **indentation** to define scope.

Trace: How the 'while' Loop Runs

Scenario: We want to print numbers from 3 down to 1.

```
int count = 3;

while (count > 0) {
    print(count);
    count--;
}
```

Iter	count > 0?	Action
1	True (3 > 0)	Print 3, count becomes 2
2	True (2 > 0)	Print 2, count becomes 1
3	True (1 > 0)	Print 1, count becomes 0
4	False (0 > 0)	Loop Terminates

Best Practice

Use a `while` loop when you **don't know** exactly how many iterations you need (e.g., waiting for user input, reading a file line-by-line).

The 'for' Loop: The Compact Workhorse

Concept: Best used when you know **exactly how many times** to repeat, or when iterating over a sequence.

C++: The "3-in-1" Header

Controls the counter manually.

```
// (Init; Condition; Update)
for (int i = 0; i < 5; i++) {
    cout << i << " ";
}
// Output: 0 1 2 3 4
```

Python: The Iterator

Visits items in a sequence.

```
# i becomes next item
automatically
for i in range(5):
    print(i, end=" ")

# Output: 0 1 2 3 4
```

Key Difference: C++ is a *manual counter*; Python is a *collection visitor*.

Python Mechanism: The Power of 'range()'

How do we translate complex C++ loops to Python?

We use arguments in `range(start, stop, step)`.

Goal	C++ Syntax	Python Syntax
Standard 0 to 9	<code>for(int i=0; i<10; i++)</code>	<code>range(10)</code>
Start from 5	<code>for(int i=5; i<10; i++)</code>	<code>range(5, 10)</code>
Step by 2	<code>for(int i=0; i<10; i+=2)</code>	<code>range(0, 10, 2)</code>
Reverse	<code>for(int i=10; i>0; i--)</code>	<code>range(10, 0, -1)</code>

Python Note

The stop value is **exclusive**.

`range(5)` goes 0, 1, 2, 3, 4. It **never** touches 5.

Scenario: Processing a List of Grades

Task: You have a list [85, 90, 78]. Print every grade.

C++ (Classic Indexing)

Access by position.

```
int grades[] = {85, 90, 78};  
  
// We need the index 'i' to  
// fetch data  
for (int i = 0; i < 3; i++) {  
    cout << grades[i] << endl;  
}
```

Python (Direct Access)

Access by value.

```
grades = [85, 90, 78]  
  
# No index needed!  
for grade in grades:  
    print(grade)
```

Evolution: The Modern C++ "Range-Based" Loop

Good News: Modern C++ (C++11 and later) borrowed this feature from Python/Java. You no longer need 'i' variables for simple lists!

Modern C++ Syntax

```
int grades[] = {85, 90, 78};  
  
// Read as: "For every 'g' in 'grades'"  
for (int g : grades) {  
    cout << g << endl;  
}
```

Why use this?

- No index math errors.
- No "off-by-one" bugs.
- Cleaner code.

Recommendation: Use this style whenever you just need to read data.

The 'do-while' Loop: Act, Then Check

Concept: The "Upside Down" loop. It executes the body **at least once** before checking the condition.

C++ / Java Syntax

Useful for Menus / Input validation.

```
int choice;
do {
    cout << "Enter 1 to Start: ";
    cin >> choice;
} while (choice != 1);
```

Risk: Condition is at the bottom, easy to miss while reading code.

Python (The Workaround)

*Python **does not** have do-while.*

```
while True:
    choice = input("Enter 1: ")

    # Manually check and break
    if choice == "1":
        break
```

Why? Python prefers explicit breaks over bottom-checks.

Note: Use sparingly. It is often safer to initialize variables correctly and use a standard `while` loop.

Loop Control: The 'break' Statement

Metaphor: The "Emergency Eject". *Used when you find what you are looking for and want to stop searching.*

```
// Scenario: Find the number 50
for (int i = 0; i < 100; i++) {

    if (i == 50) {
        break; // <--- Jumps out
    }

    cout << "Checking " << i;
}
cout << "Done.";
```

Logic Flow

- 1 Hits break.
- 2 Ignores rest of the body.
- 3 **Kills the loop entirely.**
- 4 Resumes code *after* the loop.

Loop Control: The 'continue' Statement

Metaphor: "Skip this Track". *Used to ignore specific items (like bad data) but keep the loop running.*

```
// Scenario: Print only odd numbers
for (int i = 0; i < 5; i++) {

    // If even, skip to next round
    if (i % 2 == 0) {
        continue; // <--- Jumps up
    }

    cout << i << " ";
}
// Output: 1 3
```

Logic Flow

- 1 Hits continue.
- 2 Skips lines below it.
- 3 **Stays in the loop.**
- 4 Jumps to **Update** ($i++$).

Nested Loops: The "Clock" Metaphor

Concept: A loop inside another loop. *Think of a Clock: The Minute hand must go all the way around (0-59) for the Hour hand to move just once.*

```
// Outer Loop (The "Hour" Hand)
for (int i = 0; i < 3; i++) {

    // Inner Loop (The "Minute" Hand)
    for (int j = 0; j < 2; j++) {

        cout << i << ", " << j << "  ";
    }
    cout << endl; // New line after inner
                  finishes
}
```

Visualizing the Flow

- **Outer (i):** Runs 3 times.
- **Inner (j):** Runs 2 times *per outer step*.
- **Total runs:** $3 \times 2 = 6$.

Tracing: The Multiplication Rule

Key Rule: The inner loop **resets completely** (starts over from 0) every time the outer loop steps forward.

Execution Trace

Outer (i)	Inner (j)
0	0
0	1
1	0
1	1
2	0
2	1

Performance Warning ($O(N^2)$)

If you nest two loops that both run N times, the total work is $N \times N$.

- $N = 10 \rightarrow 100$ steps.
- $N = 1,000 \rightarrow 1,000,000$ steps!

Be careful with nested loops on large data.

Let us Practice

Enough thinking.

Practice 1: Sum of N natural numbers

Source Code

```
int s = 0;
int i = 1;

while (i <= N) {
    s = s + i;
    i++;
}
printf("%d", s);
```

Analysis Tasks

- 1 **Visual:** Draw the Control Flow Graph (CFG).
- 2 **Trace:** Create a table showing i and s at each step for $N = 3$.
- 3 **Complexity:** Determine the Big O time complexity.

Solution 1: Control Flow Graph

Draw the CFG over the source code.

```
int s = 0;

int i = 1;

while (i <= N) {
    s = s + i;

    i++;
}

printf("%d", s);
```

Solution 1: Trace Table ($N = 3$)

Step	i	Check ($i \leq 3$)	Operation ($s = s + i$)	New i
Init	1	-	$s = 0$	1
Iter 1	1	True	$s = 0 + 1 \rightarrow 1$	2
Iter 2	2	True	$s = 1 + 2 \rightarrow 3$	3
Iter 3	3	True	$s = 3 + 3 \rightarrow 6$	4
Stop	4	False	Loop Terminates	-

Table: Trace table for sum of N natural numbers when N=3

Answer: 6

Solution 1: Complexity Analysis

Time Complexity

- **Analysis:** The loop runs exactly as many times as the value of N .
- **Iterations:** N iterations.
- **Operations per Iteration:** Constant (c) (one addition, one assignment, one increment).
- **Total Work:** $N \times c$.

Result: $O(N)$ (Linear)

Space Complexity

- **Analysis:** We count the memory allocated for variables.
- **Variables Used:**
 - `int s` (4 bytes)
 - `int i` (4 bytes)
 - `int N` (4 bytes)
- **Growth:** Does not change if N becomes 1,000,000.

Result: $O(1)$ (Constant)

Practice 2: The Linear Search

	0	1	2	3	4	5	6	7	8	9
Array:	18	4	6	10	1	-12	14	30	5	2

Source Code

```
int target = 1;
for (int i = 0; i < 10; i++) {
    if (arr[i] == target) {
        printf("Target at %d\n", i);
        break;
    }
    printf("Value %d\n", arr[i]);
}
printf("Done");
```

Analysis Tasks

- 1 Visual:** Draw the Control Flow Graph.
- 2 Trace:** Create a table tracking 'i' and 'Output'.
- 3 Complexity:** Analyze the complexity.

Solution 2: Control Flow Graph

Draw the CFG over the source code.

```
int target = 1;

for (int i = 0; i<10; i++) {

    if (arr[i] == target) {

        printf("Target at %d\n",i);

        break;

    }

    printf("Value %d\n", arr[i]);
}

printf("Done");
```

Solution 2: Trace Table (Target = 1)

Target: 1

Array: [18, 4, 6, 10, 1, -12...]

i	Val (arr[i])	Check (== 1)	Output	Next Step
0	18	False	"Value 18"	Continue (<i>i</i> ++)
1	4	False	"Value 4"	Continue (<i>i</i> ++)
2	6	False	"Value 6"	Continue (<i>i</i> ++)
3	10	False	"Value 10"	Continue (<i>i</i> ++)
4	1	True	"Target at 4"	BREAK
-	-	-	"Done"	Stop

Table: Trace Table for Linear Search

Note: The indices 5 through 9 are never accessed because the `break` statement terminates the loop immediately upon finding the target.

Solution 2: Complexity Analysis

Time Complexity

- **Worst Case:** Target is at the very end (index 9) or not present. Loop runs N times. ($O(N)$).
- **Best Case:** Target is at index 0. Loop runs 1 time. ($O(1)$).
- **Average Case:** Target is somewhere in the middle. $\approx N/2$ steps.

Result: $O(N)$ (Linear)

Space Complexity

- **Analysis:** We do not create copies of the array.
- **Variables:** We only use a single integer `i` for tracking the position.
- **Growth:** Constant memory usage regardless of array size.

Result: $O(1)$ (Constant)

Practice 3: Binary Search

	0	1	2	3	4	5	6	7	8	9
Array:	2	4	6	7	10	12	14	16	18	20

Source Code

```
int low = 0, high = 9;
int target = 7;
while (low <= high) {
    int mid = (low + high) / 2;
    if (arr[mid] == target) break;
    if (arr[mid] < target) low=mid+1;
    else high = mid - 1;
}
```

Analysis Tasks

- 1 Visual:** Draw the Control Flow Graph (CFG).
- 2 Trace:** Track low, high, and mid to find the value 7.
- 3 Complexity:** Analyze the time complexity of the search.

Solution 3: Control Flow Graph

Draw the CFG over the source code.

```
int low = 0, high = 9;

int target = 7;

while (low <= high) {

    int mid = (low + high) / 2;

    if (arr[mid] == target) break;

    if (arr[mid] < target) low=mid+1;

    else high = mid - 1;

}
```

Solution 3: Trace Table (Target = 7)

Array Indices:

0	1	2	3	4	5	
2	4	6	7	10	12	...

Iter	Low	High	Mid	Val	Logic Check	Next Action
1	0	9	4	10	$10 > 7$ (Too Big)	$\text{high} = 4 - 1 = 3$
2	0	3	1	4	$4 < 7$ (Too Small)	$\text{low} = 1 + 1 = 2$
3	2	3	2	6	$6 < 7$ (Too Small)	$\text{low} = 2 + 1 = 3$
4	3	3	3	7	$7 == 7$ (Match)	BREAK

Table: Trace Table for Binary Search (Finding 7)

Visual Check: By Iteration 4, 'low', 'high', and 'mid' all converge on index 3.

Solution 3: Complexity Analysis

Time Complexity

- **Mechanism:** Divide and Conquer.
- **Process:** In every step, we discard exactly half of the remaining array.
- **Math:** $N \rightarrow \frac{N}{2} \rightarrow \frac{N}{4} \rightarrow \dots \rightarrow 1$.
- This is the definition of the logarithm base 2.

Result: $O(\log N)$ (Logarithmic)

Space Complexity

- **Analysis:** This is an iterative solution (using a loop), not recursive.
- **Variables:** We only store `low`, `high`, `mid`, and `target`.
- **Growth:** Constant, regardless of array size N .

Result: $O(1)$ (Constant)

Practice 4: Bubble Sort



Source Code

```
int N = 5;
for (int i = 0; i < N-1; i++) {
    for (int j = 0; j < N-i-1; j++) {
        if (arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}
```

Analysis Tasks

- 1 Trace Pass 1:** List every comparison and indicate exactly which pairs are swapped.
- 2 Visual:** Draw the final state of the array after the outer loop finishes its first iteration ($i = 0$).
- 3 Complexity:** Derive the Big O notation based on the loop structure.

Solution 4: Trace Table (Pass 1, $i=0$)

Task: List every comparison and swap for the first pass ($i = 0$). **Input Array:** [5, 1, 4, 2, 8]

j	Pair (j, j+1)	Comparison	Action	Array State
0	(5, 1)	$5 > 1$ (True)	Swap	[1, 5, 4, 2, 8]
1	(5, 4)	$5 > 4$ (True)	Swap	[1, 4, 5, 2, 8]
2	(5, 2)	$5 > 2$ (True)	Swap	[1, 4, 2, 5, 8]
3	(5, 8)	$5 > 8$ (False)	No Swap	[1, 4, 2, 5, 8]

Table: Detailed Trace of First Outer Iteration

Solution 4: Visual Result

Task: Draw the final state of the array after the outer loop finishes its first iteration ($i = 0$).



- Notice that **5** has moved from index 0 to index 3.
- The largest element checked (8) is correctly at the end.

Solution 4: Complexity Analysis

Time Complexity

- **Logic:** We have nested loops.
- **Outer Loop:** Runs $N - 1$ times.
- **Inner Loop:** Runs $N - 1$, then $N - 2$, then $N - 3 \dots$
- **Sum:**
$$(N - 1) + (N - 2) + \dots + 1 = \frac{N(N-1)}{2}.$$
- **Approx:** $\frac{N^2}{2}.$

Result: $O(N^2)$ (Quadratic)

Space Complexity

- **Type:** In-Place Sorting.
- **Analysis:** We do not create a new array. We swap elements inside the existing one.
- **Extra Memory:** Only one integer (`temp`) is needed for the swap operation.

Result: $O(1)$ (Constant)